

Python Project

Libraries of Python Language

1.Random Module:

The random module is the python's standard library; meaning doesn't need to install it via pip. It implements pseudo-random number generators for various distributions. It is widely used in games, simulations, and basic security.

Key Functions:

- `random()`: Returns a random float between 0.0 and 1.0.
- `randint(a, b)`: Returns a random integer N such that $a \leq N \leq b$.
- `choice(sequence)`: Returns a random element from a non-empty sequence like a list or string.
- `shuffle(list)`: Reorders the elements of a list in-place.
- `sample(population, k)`: Returns a k length list of unique elements chosen from the population.

```
import random

print(f"Random float between 0 and 1: {random.random()}")

print(f"Random integer between 1 and 10: {random.randint(1, 10)}")

my_list = ['apple', 'banana', 'cherry', 'date']
print(f"Random element from list: {random.choice(my_list)}")

my_string = "Python"
print(f"Random character from string: {random.choice(my_string)}")

... Random float between 0 and 1: 0.8993611452967459
Random integer between 1 and 10: 8
Random element from list: apple
Random character from string: h
```

2. Numpy Library:

NumPy (Numerical Python) is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, called an ndarray, and tools for working with these arrays.

While Python lists can store different data types, NumPy arrays are homogeneous (all elements are the same type), which makes them significantly faster and more memory-efficient for mathematical operations.

Key functions:

- `np.array()`: The starting point for any task; it converts standard Python lists or tuples into high-performance, multi-dimensional NumPy arrays.
- `np.arange()` & `np.linspace()`: Essential for data generation; `arange` creates sequences with a specific step, while `linspace` creates a set number of evenly spaced values between two points.
- `ndarray.reshape()`: A critical tool for data preparation that allows you to change the structure of your data (e.g., turning a flat list into a matrix) without altering the actual values.

```
import numpy as np

# 1. np.array: Create a NumPy array
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
my_array = np.array(my_list)
print(f"Original array: {my_array}")

# 2. np.mean: Calculate the mean of an array
mean_value = np.mean(my_array)
print(f"Mean of the array: {mean_value}")

# 3. np.std: Calculate the standard deviation of an array
std_dev = np.std(my_array)
print(f"Standard deviation of the array: {std_dev}")

# 4. np.reshape: Change the shape of an array
reshaped_array = my_array.reshape(2, 5)
print(f"\nReshaped array (2x5):\n{reshaped_array}")

... Original array: [ 1  2  3  4  5  6  7  8  9 10]
Mean of the array: 5.5
Standard deviation of the array: 2.8722813232690143

Reshaped array (2x5):
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

- `np.mean()`, `np.median()`, & `np.std()`: These functions provide instant statistical summaries of your entire dataset or specific rows/columns with a single line of code.

3.Pandas Library:

Pandas is the most popular Python library for data analysis and manipulation. It provides high-performance, easy-to-use data structures, specifically designed to work with structured (tabular) data, much like an Excel spreadsheet or a SQL table. It is built on top of NumPy, making it incredibly fast for processing large datasets.

Key Pandas Functions:

- `pd.read_csv()` / `pd.read_excel()`: The entry point for most projects; these functions allow you to instantly load data from external files into a Pandas DataFrame.
- `df.head()` & `df.tail()`: Essential for data exploration; they allow you to quickly view the first or last few rows of your dataset to understand its structure and content.
- `df.describe()`: A powerful diagnostic tool that provides a summary of central tendency, dispersion, and shape of a dataset's distribution (mean, min, max,

etc.).

```
import pandas as pd

# Create a sample DataFrame for demonstration
data = {
    'Category': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B'],
    'Value': [10, 15, 12, 20, 18, 11, 22, 16]
}
df = pd.DataFrame(data)

print("Original DataFrame:")
display(df)
```

	Category	Value
0	A	10
1	B	15
2	A	12
3	C	20
4	B	18
5	A	11
6	C	22
7	B	16

- `df.groupby()`: Critical for data analysis; it allows you to split data into groups based on specific criteria and apply functions (like `sum` or `mean`) to each group.
- `df.fillna()` & `df.dropna()`: The primary tools for data cleaning; they allow you to handle missing or "null" values by either replacing them with specific data or removing those records entirely.

```
print("First 3 rows of the DataFrame:")
display(df.head(3))
```

First 3 rows of the DataFrame:

	Category	Value
0	A	10
1	B	15
2	A	12

```
print("Descriptive statistics of the DataFrame:")
display(df.describe())
```

Descriptive statistics of the DataFrame:

	Value
count	8.000000
mean	15.500000
std	4.342481
min	10.000000
25%	11.750000
50%	15.500000
75%	18.500000

```
print("Sum of 'Value' grouped by 'Category':")
display(df.groupby('Category')['Value'].sum().reset_index())
```

Sum of 'Value' grouped by 'Category':

	Category	Value
0	A	33
1	B	49
2	C	42

4. Matplotlib Library:

Matplotlib is the foundational library for creating static, animated, and interactive visualizations in Python. It is highly customizable and serves as the building block for many other plotting tools. Whether you need a simple line graph for a quick data check or a publication-quality figure, Matplotlib provides the flexibility to control every element of a plot, from axis labels to custom color scales.

Key Matplotlib Functions:

- `plt.plot()`: The most versatile function used to create line charts, which are ideal for visualizing trends over time or continuous data.
- `plt.scatter()`: Used to create scatter plots, which help identify correlations, clusters, or outliers between two different variables.
- `plt.bar()` / `plt.barh()`: Essential for categorical data; these functions generate vertical or horizontal bar charts to compare values across different groups.

- `plt.xlabel()`, `plt.ylabel()`, & `plt.title()`: Critical for accessibility and clarity; these functions allow you to label your axes and give your visualization a descriptive heading.
- `plt.show()`: The command that actually renders and displays the final figure on your screen after you have defined all your plotting parameters.

```
import matplotlib.pyplot as plt
import numpy as np

# --- 1. Line Plot (plt.plot) ---
# Generate data for line plot
x_line = np.linspace(0, 10, 100)
y_line = np.sin(x_line)

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1) # First subplot in a 1x3 grid
plt.plot(x_line, y_line, label='sin(x)', color='blue')
plt.title('Line Plot: sin(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.legend()

# --- 2. Scatter Plot (plt.scatter) ---
# Generate data for scatter plot
x_scatter = np.random.rand(50) * 10
y_scatter = 2 * x_scatter + np.random.randn(50) * 5

plt.subplot(1, 3, 2) # Second subplot
plt.scatter(x_scatter, y_scatter, color='green', alpha=0.7, s=50)
plt.title('Scatter Plot: Random Data')
plt.xlabel('X-value')
plt.ylabel('Y-value')
plt.grid(True)
```



5.Tkinter Library:

scripts without installing heavy third-party packages. Tkinter is Python's built-in library for creating Graphical User Interfaces (GUIs). It is the standard toolkit used to

build desktop applications with windows, buttons, menus, and text boxes. Since it comes pre-installed with Python, it is often the first choice for developers who want to create a visual interface for their scripts without installing heavy third-party packages.

Key Tkinter Functions:

- Tk(): The constructor that initializes the main application window (the "root"), which serves as the container for all other visual elements.
- Label(): A widget used to display static text or images on the screen, often used for titles, instructions, or status updates.
- Entry(): Creates a single-line text field that allows users to type in data, such as a username, a search query, or a numerical value.
- Button(): A clickable widget that triggers a specific Python function (a "callback") when pressed, making the application interactive.
- mainloop(): The most critical command; it starts an infinite event loop that keeps the window open and listens for user interactions like clicks or keystrokes.

```
import tkinter as tk
from tkinter import messagebox

# 1. Create the main application window
app = tk.Tk()
app.title("Tkinter Features Demo")
app.geometry("400x250") # Set window size

# Function to be called by the button
def on_button_click():
    user_input = entry_field.get() # Get text from entry field
    if user_input:
        messagebox.showinfo("Greeting", f"Hello, {user_input}!")
    else:
        messagebox.showwarning("Input Missing", "Please enter your name!")

# 2. Add a Label widget
label = tk.Label(app, text="Enter your name:", font=("Arial", 12))
label.pack(pady=10)

# 3. Add an Entry widget
entry_field = tk.Entry(app, width=30, font=("Arial", 12))
entry_field.pack(pady=5)

# 4. Add a Button widget
button = tk.Button(app, text="Say Hello", command=on_button_click, font=("Arial", 12))
button.pack(pady=10)
```

6. Math Library:

The math module is a built-in Python library that provides access to the mathematical functions defined by the C standard. It is essential for performing complex mathematical operations that go beyond basic arithmetic, such as trigonometry, logarithms, and power functions. Unlike NumPy, which is designed for arrays, the math library is optimized for scalar (single) values and provides high precision.

Key Math Functions:

- `math.sqrt(x)`: Returns the square root of `x`. It is more efficient than using `x**0.5` for single numerical values.
- `math.ceil()` & `math.floor()`: Used for rounding; `ceil` rounds a number up to the nearest integer, while `floor` rounds it down.
- `math.pow(x, y)`: Raises `x` to the power of `y`. Unlike the `**` operator, `math.pow` always converts its arguments to floats and returns a float.
- `math.pi` & `math.e`: These aren't functions but mathematical constants. They provide the most accurate representation of π and e available in Python.

```
import math
number = 25
sqrt_result = math.sqrt(number)
print(f"Square root of {number}: {sqrt_result}")

decimal_up = 4.2
ceil_result = math.ceil(decimal_up)
print(f"Ceiling of {decimal_up}: {ceil_result}")

decimal_down = 4.8
floor_result = math.floor(decimal_down)
print(f"Floor of {decimal_down}: {floor_result}")

base = 2
exponent = 3
pow_result = math.pow(base, exponent)
print(f"{base} raised to the power of {exponent}: {pow_result}")

fact_number = 5
factorial_result = math.factorial(fact_number)
print(f"Factorial of {fact_number}: {factorial_result}")

Square root of 25: 5.0
Ceiling of 4.2: 5
Floor of 4.8: 4
2 raised to the power of 3: 8.0
Factorial of 5: 120
```

7. OS Library:

The `os` module is a built-in Python library that provides a portable way of using operating system-dependent functionality. It allows your Python scripts to interact directly with the file system and the underlying OS. Whether you are building an automated file organizer, a data processing pipeline, or a backup script, the `os` library is the standard tool for managing directories, checking file paths, and handling environment variables.

Key OS Functions:

- `os.getcwd()`: Short for "Get Current Working Directory," this function returns the absolute path of the folder where your script is currently running.
- `os.listdir(path)`: Returns a list containing the names of the entries (files and folders) in the directory given by the path.
- `os.mkdir()` / `os.makedirs()`: Used to create new directories; `makedirs` is particularly useful as it can create a whole nested path of folders at once.
- `os.path.join()`: A critical function for cross-platform compatibility; it intelligently joins folder and file names using the correct slash (/ or \) for the user's operating system.
- `os.path.exists()`: A safety check that returns `True` if a specific file or directory already exists, preventing errors before trying to open or move a file.

```
import os

# 1. os.getcwd(): Get the current working directory
current_directory = os.getcwd()
print(f"Current working directory: {current_directory}")

# 2. os.listdir(): List contents of a directory
# This will list files and directories in the current working directory
print(f"Contents of current directory: {os.listdir(current_directory)}")

# 3. os.path.join(): Join path components intelligently
file_name = "my_document.txt"
full_path = os.path.join(current_directory, file_name)
print(f"Joined path: {full_path}")

... Current working directory: /content
Contents of current directory: ['.config', 'my_new_directory', 'sample_data']
Joined path: /content/my_document.txt
```


8. The requests Library:

Requests is the "HTTP for Humans" library. It is the industry standard for making web requests, allowing your Python scripts to interact with websites and APIs. It simplifies the process of sending data to servers or pulling data down for analysis, handling complex tasks like authentication and cookies automatically.

Key Requests Functions:

- `requests.get(url)`: The most common function; it retrieves data from a specified web address.
- `requests.post(url, data)`: Sends data to a server, typically used for submitting forms or uploading files.
- `response.json()`: A built-in decoder that converts a JSON response from an API directly into a Python dictionary.
- `response.status_code`: Returns a number (like 200 for success or 404 for not found) to tell you if the request worked.
- `requests.Session()`: Allows you to persist certain parameters (like login credentials) across multiple requests to the same website.

```
import requests
# 1. requests.get(): Make a GET request
# This fetches data from a specified URL.
try:
    response_get = requests.get('https://api.github.com/events')
    print(f"GET Request Status Code: {response_get.status_code}")
# 2. Access JSON content
    if response_get.status_code == 200:
        print(f"First event type (from JSON): {response_get.json()[0]['type']}")
except requests.exceptions.RequestException as e:
    print(f"GET Request Error: {e}")

# 3. requests.post(): Make a POST request
# This sends data to a specified URL.
# Using a dummy API for demonstration.
post_data = {'title': 'foo', 'body': 'bar', 'userId': 1}
try:
    response_post = requests.post('https://jsonplaceholder.typicode.com/posts', json=post_data)
    print(f"\nPOST Request Status Code: {response_post.status_code}")
    if response_post.status_code == 201:
        print(f"POST Response (from JSON): {response_post.json()}")
except requests.exceptions.RequestException as e:
    print(f"POST Request Error: {e}")

GET Request Status Code: 200
First event type (from JSON): PushEvent

POST Request Status Code: 201
POST Response (from JSON): {'title': 'foo', 'body': 'bar', 'userId': 1, 'id': 101}
```

9. The json Library:

JSON (JavaScript Object Notation) is the most common format for exchanging data on the web. The built-in json library allows Python to parse JSON data into dictionaries and lists, and vice-versa. This is critical for saving configuration files, storing data, or communicating with modern web applications.

Key JSON Functions:

- `json.loads()`: Takes a JSON string and converts it into a Python dictionary.
- `json.dumps()`: Takes a Python dictionary and converts it into a JSON-formatted string.
- `json.load()`: Reads JSON data directly from a file.
- `json.dump()`: Writes Python data structures directly into a JSON file.
- `Indent = parameter`: Used within `dumps` or `dump` to make the output "pretty" and readable by adding spacing.

```
json.dump(python_dict, f, indent=4)
print(f"Python dictionary saved to '{file_name}'")

with open(file_name, 'r') as f:
    loaded_from_file = json.load(f)
print(f"Python dictionary loaded from '{file_name}': {loaded_from_file}")

Original Python dictionary: {'name': 'Alice', 'age': 30, 'isStudent': False, 'courses': [{'title': 'Hist
Type of original Python dictionary: <class 'dict'>

JSON formatted string:
{
    "name": "Alice",
    "age": 30,
    "isStudent": false,
    "courses": [
        {
            "title": "History",
            "credits": 3
        },
        {
            "title": "Math",
            "credits": 4
        }
    ]
}
Type of JSON string: <class 'str'>
```

```
Type of JSON string: <class 'str'>

Loaded Python dictionary from JSON string: {'name': 'Alice', 'age': 30, 'isStudent': False, 'courses': []}
Type of loaded Python dictionary: <class 'dict'>

Python dictionary saved to 'data.json'
Python dictionary loaded from 'data.json': {'name': 'Alice', 'age': 30, 'isStudent': False, 'courses': []}
```

10. The datetime Library:

The datetime module is the standard way to handle dates, times, and intervals in Python. It is indispensable for any project involving logs, scheduling, or time-series data (like financial trends or project deadlines).

Key DateTime Functions:

- `datetime.now()`: Returns the current local date and time.
- `strftime()`: Formats a date object into a readable string (e.g., "2026-04-30").
- `strptime()`: The reverse of `strftime`; it takes a string and turns it into a date object.
- `timedelta()`: Represents a duration (e.g., "7 days") and is used to perform "date math" (like adding 5 days to today).
- `date.today()`: A simpler function that returns only the current date without the time.

```
import datetime

# 1. datetime.datetime.now(): Get the current date and time
current_dt = datetime.datetime.now()
print(f"Current date and time: {current_dt}")

# 2. datetime.datetime.strftime(): Format a datetime object into a string
formatted_dt = current_dt.strftime("%Y-%m-%d %H:%M:%S")
print(f"Formatted date and time: {formatted_dt}")

# 3. datetime.datetime.strptime(): Parse a string into a datetime object
date_string = "2023-10-27 10:30:00"
parsed_dt = datetime.datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print(f"Parsed date and time from string: {parsed_dt}")

Current date and time: 2026-04-30 11:01:39.021889
Formatted date and time: 2026-04-30 11:01:39
Parsed date and time from string: 2023-10-27 10:30:00
```